

Z0237282 : Medium Bonds & Quantile Regression & Shock_log_dev

1 Introduction

The bond market is crucial for government financing, yet entails risks, profits, and losses for direct participants (dealers, investors, government), while indirectly affecting citizens and overall market stability. Literature highlights concerns regarding the public costs of bond issuance's effects on prices and yields, prompting a market microstructure analysis of liquidity.

We apply descriptive statistics, graphical analysis, quantile regression, and machine learning techniques to identify data patterns and examine heterogeneous shock impacts across the distribution.

Since establishing causality would require richer data and more extensive analysis, our interpretations and deductions remain preliminary and are presented mainly for simplicity and learning purposes, with full awareness that any interpretation must be made cautiously.

We include VSTOXX (VIX_Euro) data for our sample dates to capture European market stress. Being comfortable with R, we use Python only for graphical analysis. We provide commented and replicable code in the appendix.

2 Descriptive statistics and data structure

2.1 Descriptive statistics

Table 1: Descriptive Statistics

Variable	Count	Mean	Std	Min	25%	50%	75%	Max	Missing	Skew	Kurtosis
Years_to_maturity	342.00	6.76	1.18	5.07	5.79	6.46	7.68	9.49	0	0.57	-0.79
Amount_offered_min	342.00	7532.75	1929.13	3000.00	6500.00	7500.00	9000.00	11500.00	0	-0.15	-0.24
Amount_offered_max	342.00	8747.66	2124.42	3500.00	7500.00	8500.00	10500.00	13500.00	0	-0.04	-0.36
Bid_amount	342.00	6213.34	2755.32	1340.00	4230.00	5740.50	7556.25	21732.00	0	1.61	4.83
Amount_served	342.00	2615.25	1157.04	595.00	1685.00	2494.00	3382.75	7674.00	0	0.75	0.98
Bid_to_cover	342.00	2.50	0.71	1.20	2.03	2.38	2.71	5.77	0	1.55	3.15
Outstanding_notional	342.00	328.20	425.47	0.00	0.00	88.00	607.25	2597.00	0	1.40	2.35
Total_amount_issued	342.00	2943.46	1380.74	595.00	1813.75	2720.00	3906.75	10095.00	0	0.92	1.79
Vix_Euro	342.00	21.71	8.85	11.01	16.63	19.53	24.60	78.35	0	2.87	13.18
Auction_yield	342.00	1.18	1.37	-0.70	-0.06	0.76	2.64	4.21	0	0.33	-1.38
Weighted_average_price	342.00	1.04	0.10	0.79	0.99	1.02	1.08	1.52	0	0.91	2.88
Benchmark_yield	342.00	1.00	1.34	-0.80	-0.30	0.70	2.25	4.29	0	0.38	-1.14
Auction_premium	342.00	0.19	0.24	-0.65	0.07	0.20	0.33	1.06	0	-0.43	1.02
Auction_premium_bp	342.00	18.53	24.47	-64.70	6.54	19.85	32.91	105.72	0	-0.43	1.02
Shock_outstanding	193.00	29.34	135.64	3.83	4.31	4.73	8.20	1251.00	149	7.71	60.94
Shock_log_dev	342.00	-0.00	0.49	-1.49	-0.37	0.03	0.40	1.35	0	-0.33	-0.32
Shock_rolling	340.00	0.01	0.40	-0.70	-0.29	-0.02	0.28	1.52	2	0.74	0.80
Log_total_amount_issued	342.00	7.87	0.49	6.39	7.50	7.91	8.27	9.22	0	-0.33	-0.32
Log_outstanding_notional	193.00	5.92	1.25	0.00	5.52	6.27	6.77	7.86	149	-1.97	5.31

Note: We multiplied auction–yield by 100 for harmonization with the benchmark–yield .

Our sample (20-09-2007 to 20-10-2025) comprises medium-term bonds with a 6.76-year average maturity. The average bid-to-cover ratio is 2.5. Its distribution (positive skewness, high kurtosis) is primarily demand-driven: bid amounts exhibit stronger skewness and kurtosis than amounts served, driving extreme bid-to-cover fluctuations. This ratio appears temporally stable: (Beetsma et al., 2018a) report a 2.78 average for French medium-term bonds (1999–2014). However, cross-country heterogeneity is substantial (e.g., German averages remain below 2), suggesting our results may not generalize to regions with differing liquidity and market behaviors. US patterns are similar, averaging 2.57 between 1980 and 2008 (Lou

et al., 2013).

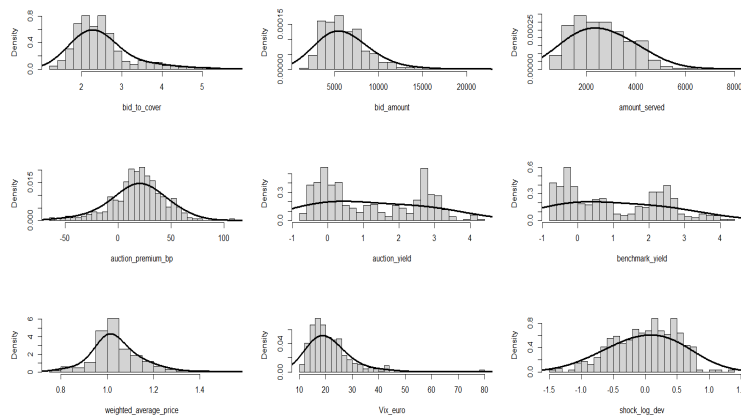
While 75% of VixEuro observations remain below 25 points (calm periods), the sample captures substantial turbulence (maximum 78.35). High kurtosis (13.18) and positive skewness confirm extreme upward spikes during major stress episodes like the 2008 financial crisis and COVID-19.

The positive average auction premium (0.19) indicates new bonds are issued at higher yields than comparable secondary market bonds, reflecting auction price pressure : dealers require compensation despite institutional bargaining power (Foucault et al., 2013). The average auction yield (1.18) also suggests a substantial increase after 2014, the end of the period covered by Beetsma et al. (2018a), who report an average of only 0.54. Furthermore, our 18.5 basis point average premium is relatively high versus the US (under 5 basis points for 5- and 10-year maturities) (Lou et al., 2013).

However, the positive skewness and relatively high kurtosis (2.88) of the weighted average price, alongside the bid amount's high skewness (1.61), suggest rare aggressive bidding. This likely reflects dealers' strategic behavior when they are particularly interested in some auctions.

Our shock measure, Shock-log-dev, centers around zero with a 0.49 standard deviation. Its near-zero skewness and kurtosis suggest the log transformation minimizes outlier influence, and may therefore produce more stable and reliable regression results.

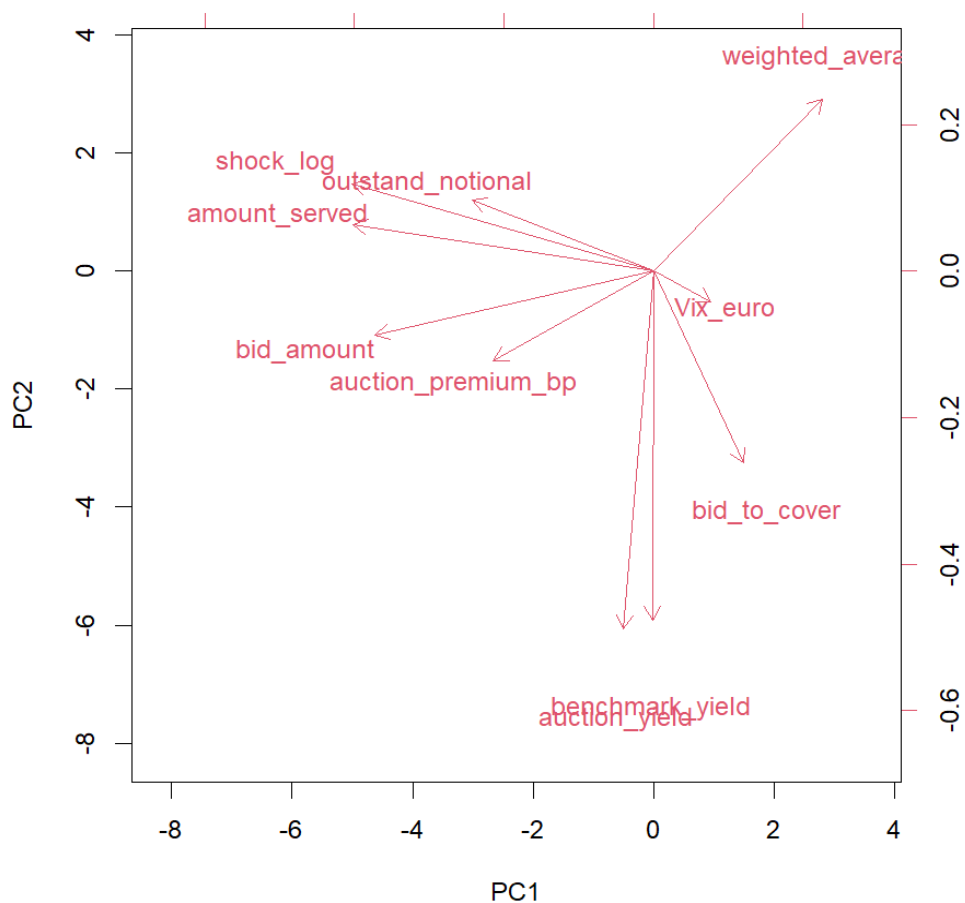
Figure 1: Distribution of variables



2.2 Data structure : machine learning techniques using unsupervised methods

To capture complex agent interactions, we use machine learning—specifically, principal component analysis (PCA). As an unsupervised method requiring no outcome variable, PCA reveals underlying relationships by summarizing correlated variables into common components, enabling us to explore how agents interact.

Figure 2: Two first components



Note: We used standardized scale to obtain stable outcomes and include only relevant variables .

Arrow length reflects how well components represent a variable. Arrows parallel to an axis (PC1/PC2) indicate strong correlation with that component, unlike orthogonal ones. Between variables, small angles imply strong positive correlation, opposite directions (180°) indicate strong negative correlation, and orthogonal arrows suggest weaker relationship rel-

ative to the other variables. Consequently, three distinct clusters emerge:

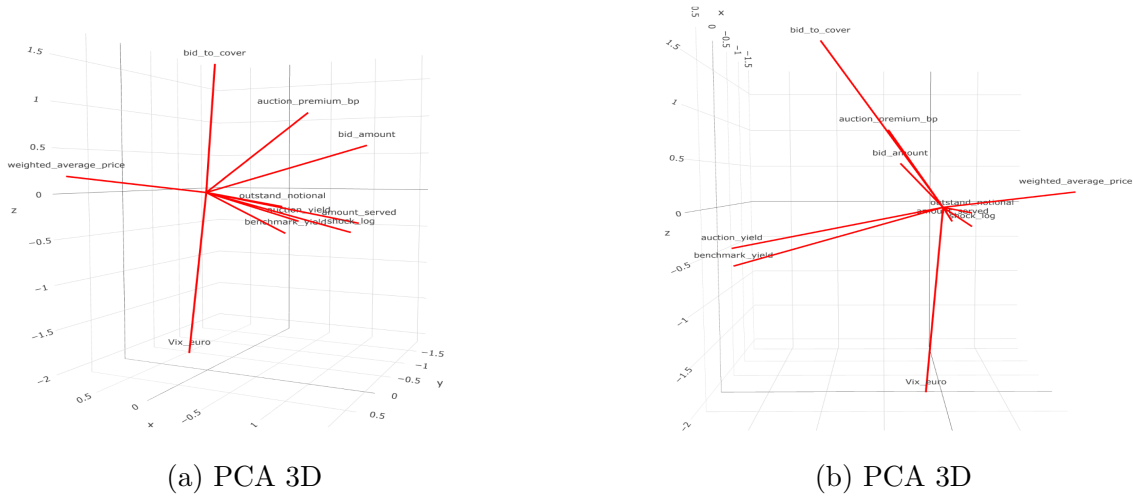
1. **Institutional component (strongly correlated with PC1):** Shock_log, outstanding_notional, and amount_served. As the first principal component (PC1) captures the most data variation, this cluster indicates issuance-related variables drive significant overall market variation. Consequently, government bond issuance shocks quantitatively outweigh other agents' actions, reflecting the sovereign issuer's strong institutional market role.
2. **Market component and outcomes (strongly correlated with PC2):** Benchmark_yield, auction_yield, and bid_to_cover. This cluster's near-orthogonality to the institutional cluster suggests issuance decisions may be disconnected from prevailing market conditions, driven instead by government financing needs. Because benchmark yields influence new auction yields, this behavior raises public borrowing efficiency and accountability concerns, as citizens ultimately bear these costs. The issuer may resemble a liquidity-driven agent, as discussed in Lecture 6 ([Karam, 2026](#)). Also, the bid-to-cover ratio shifts slightly rightward relative to that cluster, moving inversely to the institutional cluster. This reflects its negative correlation with bond supply: ceteris paribus, higher issuance reduces the ratio.
3. **liquidity Supply and outcomes (correlated with both PC1 and PC2):** bid_amount, weighted_average_price and premia. As dealers provide liquidity post-issuance, this cluster's position between the institutional and market clusters may indicate strategic behavior: bids reflect both issuance quantity risks and market-implied expected profits. Price is inverse to bid and premia following the economic logic : higher price of bonds discourage liquidity suppliers and leads to less premia.

Premia proximity to bid_amount implies liquidity suppliers strongly dictate the premium, despite the government selecting offers.

Ultimately, while institutional and market factors (PC1/PC2) define the environment, dealers focus on premia, where their influence remains substantial.

The VIX is not well explained by PC1 and PC2 (short arrow). We therefore present a 3D representation in two orientations.

Figure 3: 3D PC plots

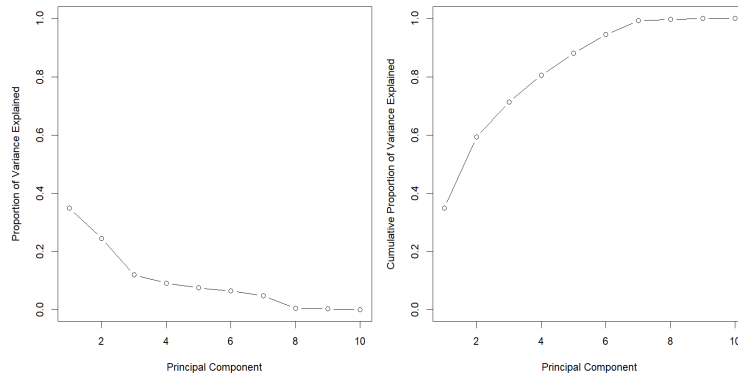


Note: The PCA is based on standardized variables to ensure a stable and reliable interpretation.

The liquidity supply cluster points in the opposite direction to the Vix, suggesting that dealers may supply less liquidity during periods of stress and receive lower premiums. The bid.to.cover variable appears closely aligned with the VIX, suggesting that dealers adjust their bids in a way that reflects the market uncertainty when determining the supply–demand equilibrium.

Across all three preceding figures, the ' premia ' arrow is medium-length and oblique to all PC reflecting it is a composite result of the whole economic environment.

Figure 4: Cumulative variation explained



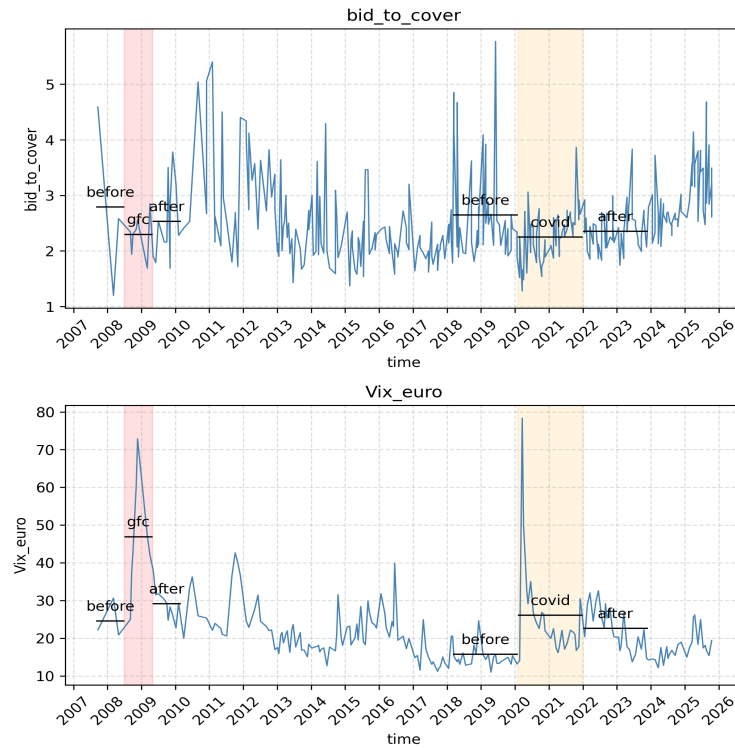
The first three PCs (reflecting issuance behaviour, liquidity provision, market conditions, and uncertainty) explain approximately 75% of total variation, thereby capturing most of

the environment's structure.

2.3 Graphical Analysis

We followed literature to define the financial crisis and COVID periods, which are main crisis in our dataset.

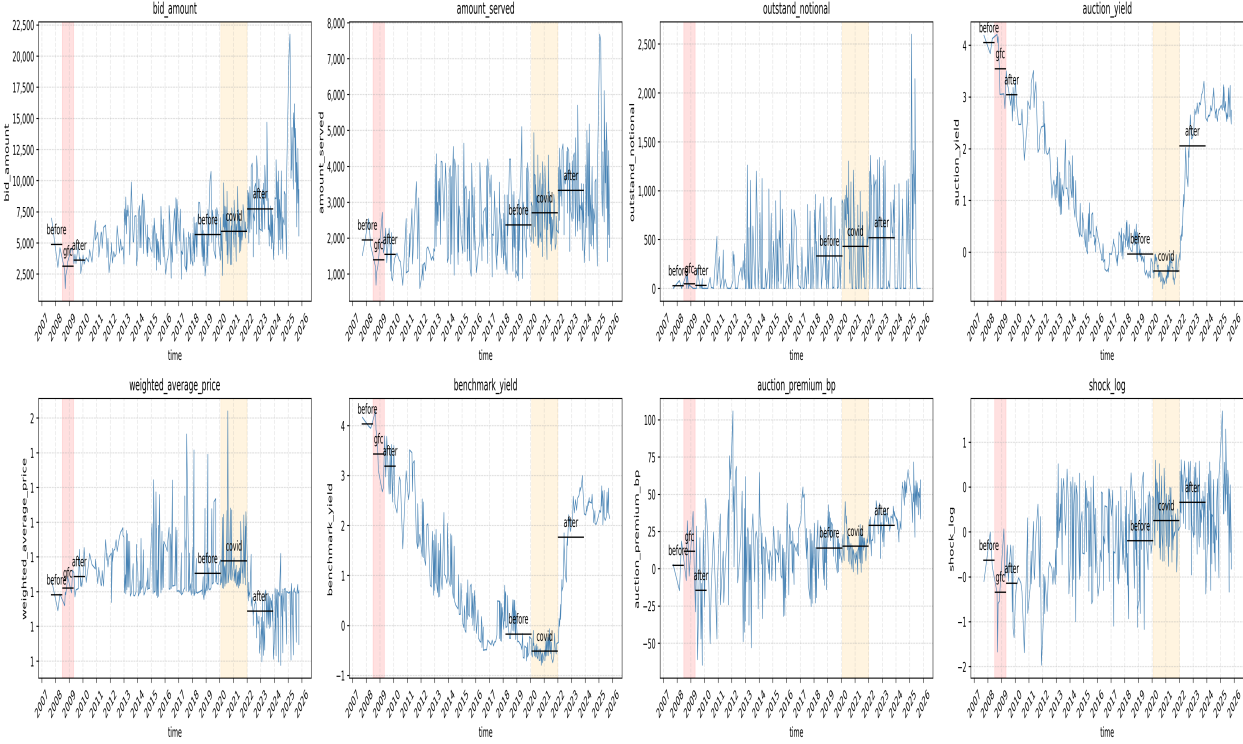
Figure 5: Bid-to-cover across different market regimes



During crisis periods, less liquidity is available to absorb issuance. Consistent with [Ferrara \(2024\)](#), higher market volatility increases dealers' risk exposure, inducing cautious intermediation. Similarly, [Foucault et al. \(2013\)](#)'s inventory holding cost model shows that fundamental volatility dictates dealers' willingness to manage inventories for market liquidity; accordingly, heightened financial crisis risks reduced their inventory holdings ([Foucault et al., 2013](#)).

The figure below reveals no systematic pattern between crises and other variables, suggesting dealers adjust mainly bids by balancing supply-demand conditions against risk levels.

Figure 6: Other variables



3 Quantile Regression

We will perform :

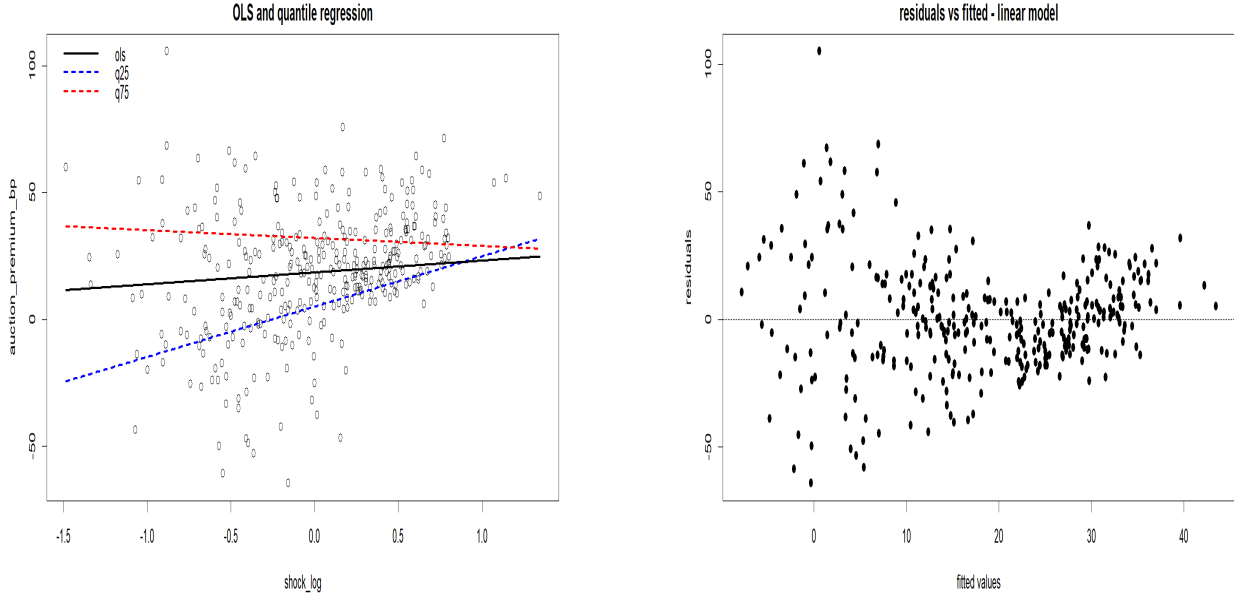
$$Q_{\tau}(Y_t | X_t) = \alpha_{\tau} + \beta_{\tau} Shock_log_dev_t \quad (1)$$

$Q_{\tau}(Y_t | X_t)$ is the τ -th quantile; Y_t the auction outcome; X_t controls; β_{τ} the shock effect.

We will control for all variables available when the shock is issued that may act as potential confounders. However, we will not control for mediators, that is, variables affected by the shock itself. This is because the market microstructure environment is shaped by strategic interactions among agents, and we want these interactions to be reflected in our outcome model. Controlling for such variables would remove part of the total effect of the shock on the outcome, so β would capture only a partial effect.

$$Y_t = \alpha_{\tau} + \beta_{\tau} Shock_log_dev_t + \gamma_{1,\tau} Vix_euro_t + \gamma_{2,\tau} outstand_notional_t + \gamma_{3,\tau} auction_date_t + \epsilon_t$$

Figure 7: Quantile fit



Overall, the left graph shows that higher shock results in higher premia. Visually, the OLS line does not fit the data well, as the pattern appears complex. In the right figure, although the residuals seem approximately normal, there is some suspicion of heteroscedasticity, since their variability appears much larger on the left side of the right figure.

Table 2: Breusch–Pagan Test for Heteroskedasticity

Test	BP statistic	df	p-value
Studentized Breusch–Pagan	58.833	4	$5.101e - 12$

The BP test strongly rejects the null of homoskedasticity ($p < 0.001$), indicating the presence of heteroskedasticity. Then, we regress with Robust estimations.

Table 3: Quantile and OLS

	dependent variable: auction_premium_bp				
	Linear quantile regression		OLS		Quadratic regression
	(25%)	(75%)			
	(1)	(2)	(3)	(4)	(5)
shock_log	19.854***	−3.095	13.297***	4.670	10.569***
	(2.643)	(3.924)	(3.625)	(3.536)	(2.876)
<i>Shock_log</i> ²					19.991***
					(4.204)
Vix_euro	−0.015	0.261	−0.118	0.140	0.106
	(0.130)	(0.212)	(0.121)	(0.130)	(0.120)
outstand_notional	−0.002	0.003	0.003	0.002	−0.004*
	(0.002)	(0.003)	(0.003)	(0.003)	(0.002)
auction_date	0.007***	0.004***		0.006***	0.006***
	(0.001)	(0.001)		(0.001)	(0.001)
constant	−116.461***	−52.290***	20.150***	−89.638***	−94.515***
	(16.084)	(16.634)	(2.811)	(15.004)	(14.777)
R-squared			0.094	0.223	0.275
Adjusted R-squared			0.086	0.214	0.264
Observations	342	342	342	342	342

note: * $p < 0.1$; ** $p < 0.05$; *** $p < 0.01$

In the 25th quantile regression (1), the coefficient (19.854) is positive and highly significant (1% level), linking increased issuance to higher premia in the lower distribution tail. Conversely, the 75th quantile coefficient is insignificant, indicating no clear supply effect on upper-tail premia.

Time controls are essential to prevent spurious relationships, given the natural upward trend in both bond supply and auction premia. Without time controls, OLS (3) captures a spurious significant effect. Including time controls in (4) renders the linear quantity-premia relationship insignificant. The VIX is also insignificant, suggesting the previously reported

PCA relationship is weak and interdependent.

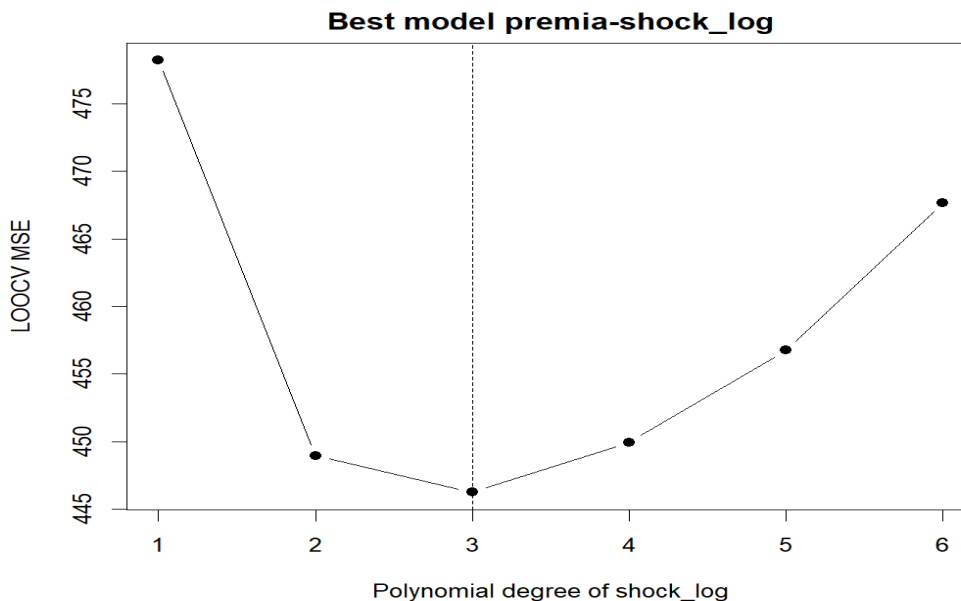
However, the quadratic specification (5) outperforms the linear model (higher R^2). Introducing a squared shock term makes both linear and quadratic terms significant, revealing a coherent, nonlinear effect of bond issuance on premia. Consequently, our linear quantile regressions may suffer from functional form misspecification.

We investigate this nonlinearity using machine learning to re-estimate appropriate quantile regressions.

3.1 Machine learning : splines and local regression methods

To maintain interpretability and avoid a black-box, we use the simplest spline-based specification: polynomials. We apply leave-one-out cross-validation to select the model that minimizes the mean squared error (MSE).

Figure 8: Best specification



The figure confirms a polynomial fit (degree 3) minimizes errors compared to the linear specification. This aligns with lecture discussions on Kyle's lambda's limitations, which is too restrictive to capture demand pressure and nonlinear bond market behavior. It also matches the square-root market impact formula confirmed for options [Tóth et al. \(2016\)](#). Again, this raises concerns about the public losses incurred in that market.

Because degree 2 and 3 MSEs are nearly identical, we estimate a quadratic quantile model (degree 2) for better interpretability.

Table 4: Quadratic Quantile and OLS with Robust Standard Errors

	Dependent variable : auction_premium_bp		
	Quantile Regression		Quadratic OLS (robust SE)
	(25%)	(75%)	
Shock_log	21.072*** (3.620)	0.971 (4.451)	10.569*** (2.876)
<i>Shock_log</i> ²	8.971 (5.965)	18.859*** (7.017)	19.991*** (4.204)
Vix_euro	0.038 (0.081)	0.222 (0.225)	0.106 (0.120)
outstand_notional	-0.006** (0.003)	-0.004 (0.004)	-0.004* (0.002)
auction_date	0.007*** (0.001)	0.005*** (0.001)	0.006*** (0.001)
Constant	-118.169*** (17.927)	-61.198*** (13.321)	-94.515*** (14.777)
Observations	342	342	342

Note: * $p < 0.1$; ** $p < 0.05$; *** $p < 0.01$.

Quadratic OLS (3) reveals a non-linear shock effect on auction premia, as both linear and squared terms are significant. However, quantile regressions demonstrate this relationship varies across the distribution.

In the lower tail (low premia), only the linear coefficient is significant: a one-unit shock increase yields a 21-point premia rise. The insignificant and small squared term implies pricing pressure remains proportional to the shock.

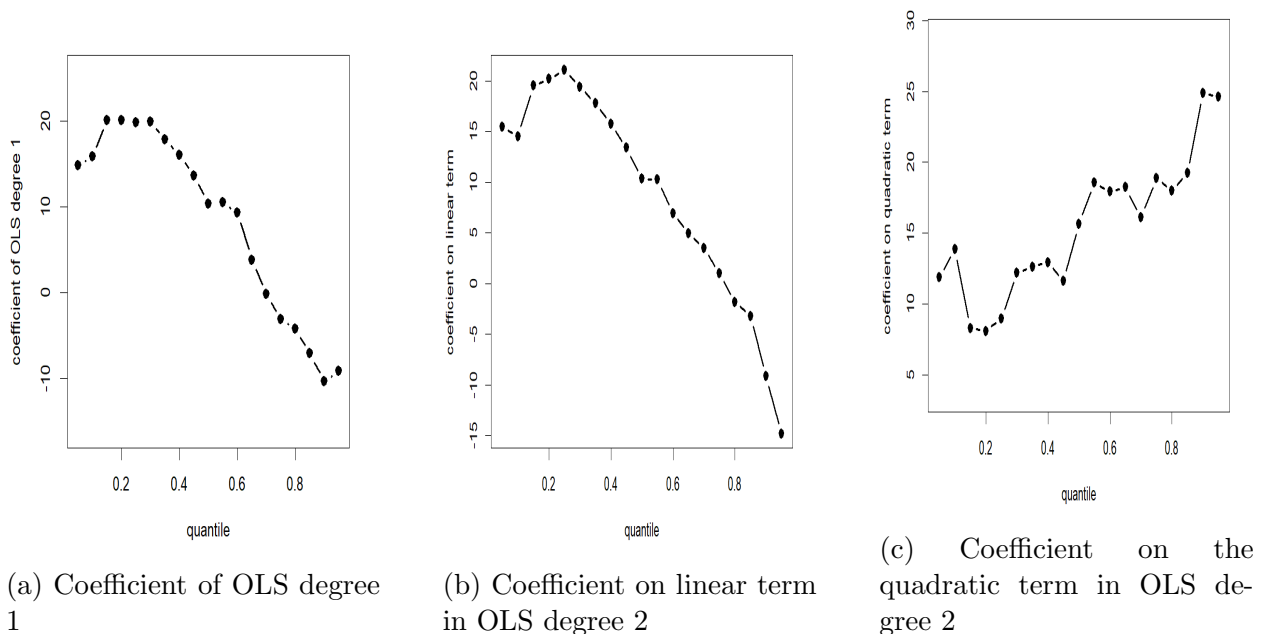
Conversely, in the upper tail (high premia), the linear term approaches zero and loses significance, while the squared term becomes strongly significant. This pronounced non-

linearity suggests large issuance shocks interact with constrained dealer balance-sheets and reduced intermediation capacity, heightening pricing pressure sensitivity. Consequently, dealers demand higher risk compensation. This aligns with [Beetsma et al. \(2018b\)](#), who notes primary dealers face tighter risk-bearing constraints when absorbing new issues, and [Ferrara et al. \(2021\)](#), who links bond price volatility to auction premia. These findings raise concerns about dealer intermediation capacity.

4 Robustness check

Quantile outcome

Figure 9: Evolution of distribution across quantile



To assess robustness across tails, we examine coefficient evolution across the entire quantile distribution. Both linear and quadratic specifications exhibit strong, yet directionally divergent, heterogeneity.

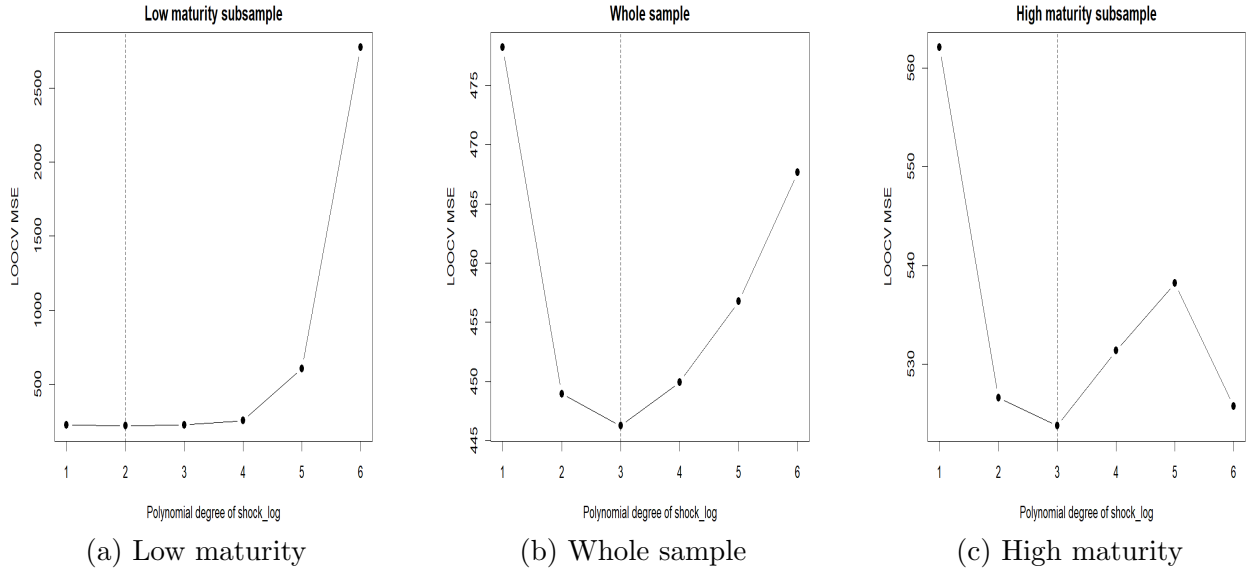
The linear model (a) which is not the appropriate specification exhibit stronger lower-tail effects. The quadratic model's linear term (b) displays the same pattern whereas the

quadratic term (c) shows inventory shocks dominating the upper tail. Because this term is squared, moderate coefficients yield substantial marginal effects that can drive the overall direction. Ultimately, these conflicting directional effects across specifications warrant caution, as results may not hold uniformly.

Robustness across Maturities

Using machine learning (Leave-One-Out CV), we test whether results hold across maturities by splitting our sample into two 30% extremes : the shortest (near 5 years, potentially informing shorter-maturity bonds) and the longest maturities.

Figure 10: Best models



Lower-maturity bonds (a) exhibit distinct patterns: while quadratic models fit best, negligible MSE differences at degree 1 suggest linear also fits well. Economically, these less duration-sensitive bonds are easier for dealers to absorb, yielding a near-linear price-quantity relationship.

Conversely, the full sample and longest-maturity bonds require a degree-3 polynomial and show substantial MSE improvements from degree 1 to 2. Exposed to higher inventory risk and balance-sheet intensity, longer-maturity bonds elicit complex dealer responses and non-linear auction outcomes. Consequently, baseline findings may vary across maturity segments.

5 Discussion and Conclusion

Although preliminary, our analysis suggests that government bond issuance is driven by financing needs rather than immediate market risk. Dealers leverage their market power to adjust bids, demanding auction premia as compensation for inventory risk. Higher market uncertainty reduces dealer liquidity rather than directly increasing premia; consequently, greater bond supply raises government financing costs, a burden ultimately borne by citizens. This relationship is non-linear and heterogeneous across quantiles. While dealers may remain profitable, severe crises reduce their intermediation capacity rather than maximizing profits. Medium-term bonds, however, remain relatively safe for investors. Overall, concerns about dealer intermediation capacity and the efficiency of public borrowing may justify monetary policy interventions, as studies ([Ferrara et al., 2021](#); [Arrata and Nguyen, 2017](#)) have shown that these measures can reduce bond yields and volatility.

References

- William Arrata and Benoît Nguyen. Price impact of bond supply shocks: Evidence from the eurosystem’s asset purchase program. 2017.
- Roel Beetsma, Massimo Giuliadori, Jesper Hanson, and Frank De Jong. Bid-to-cover and yield changes around public debt auctions in the euro area. *Journal of Banking and Finance*, 87:118–134, 2018a.
- Roel Beetsma, Massimo Giuliadori, Jesper Hanson, and Frank De Jong. Cross-border auction cycle effects of sovereign bond issuance in the euro area. *Journal of Money, Credit and Banking*, 50(7):1401–1440, 2018b.
- Jens Dick-Nielsen, Peter Feldhütter, and David Lando. Corporate bond liquidity before and after the onset of the subprime crisis. *Journal of Financial Economics*, 103(3):471–492, 2012.
- Federico M Ferrara. *Central bank asset purchases and auction cycles revisited: New evidence from the euro area*. Number 2927. ECB Working Paper, 2024.

Gerardo Ferrara, Maria Flora, and Roberto Renò. The covid-19 auction premium. Technical report, Working paper, 2021.

Thierry Foucault, Marco Pagano, and Ailsa Röell. *Market Liquidity: Theory, Evidence, and Policy*. Oxford University Press, 2013.

Arzé Karam. Market microstructure. *Durham University Business School*, 2026. Teaching material.

Dong Lou, Hongjun Yan, and Jinfan Zhang. Anticipated and repeated shocks in liquid markets. *The Review of Financial Studies*, 26(8):1891–1912, 2013.

Bence Tóth, Zoltán Eisler, and J-P Bouchaud. The square-root impace law also holds for option markets. *Wilmott*, 2016(85):70–73, 2016.

6 Appendix : Codes

Python codes for graphs

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from matplotlib.ticker import StrMethodFormatter
import math

date_col = "auction_date"
variables_to_plot = ["bid_amount", ]
n_rows = 2
n_cols = 5
subplot_width = 7
subplot_height = 4
date_format = "%m/%d/%Y"

# 2. data preparation
df[date_col] = pd.to_datetime(df[date_col], format=date_format, errors="coerce")
for var in variables_to_plot:
    df[var] = pd.to_numeric(df[var], errors="coerce")
df = df.dropna(subset=[date_col]).copy()
df = df.sort_values(date_col)

# 3. define periods
gfc_start = pd.Timestamp("2008-07-01")
gfc_end = pd.Timestamp("2009-04-30")
```



```

covid_start = pd.Timestamp("2020-02-02")
covid_end   = pd.Timestamp("2021-12-31")
def same_length_windows(start, end):
    n_days = (end - start).days + 1
    before_end = start - pd.Timedelta(days=1)
    before_start = before_end - pd.Timedelta(days=n_days - 1)
    after_start = end + pd.Timedelta(days=1)
    after_end = after_start + pd.Timedelta(days=n_days - 1)
    return before_start, before_end, after_start, after_end
gfc_before_start, gfc_before_end, gfc_after_start, gfc_after_end = same_length_windows(gfc_start, gfc_end)
covid_before_start, covid_before_end, covid_after_start, covid_after_end = same_length_windows(covid_start, covid_end)
# 4. period dummies / boolean indicators
df["bool_gfc_before"] = df[date_col].between(gfc_before_start, gfc_before_end)
df["bool_gfc_during"] = df[date_col].between(gfc_start, gfc_end)
df["bool_gfc_after"] = df[date_col].between(gfc_after_start, gfc_after_end)
df["bool_covid_before"] = df[date_col].between(covid_before_start, covid_before_end)
df["bool_covid_during"] = df[date_col].between(covid_start, covid_end)
df["bool_covid_after"] = df[date_col].between(covid_after_start, covid_after_end)
# 5. function to compute mean
def get_mean(data, var, bool_col):
    return data.loc[data[bool_col], var].mean()
fig, axes = plt.subplots(
    n_rows,
    n_cols,
    figsize=(n_cols * subplot_width, n_rows * subplot_height)
)
if n_rows == 1 and n_cols == 1:
    axes = [axes]
else:
    axes = axes.flatten()
for i, var in enumerate(variables_to_plot):
    if i >= len(axes):
        break
    ax = axes[i]
    temp = df.dropna(subset=[var]).copy()
    # main line
    ax.plot(temp[date_col], temp[var], color="steelblue", linewidth=0.9)
    # shaded areas
    ax.axvspan(gfc_start, gfc_end, color="red", alpha=0.12)
    ax.axvspan(covid_start, covid_end, color="orange", alpha=0.12)
    # means
    mean_gfc_before = get_mean(temp, var, "bool_gfc_before")
    mean_gfc_during = get_mean(temp, var, "bool_gfc_during")
    mean_gfc_after = get_mean(temp, var, "bool_gfc_after")
    mean_covid_before = get_mean(temp, var, "bool_covid_before")

```

```

mean_covid_during = get_mean(temp, var, "bool_covid_during")
mean_covid_after = get_mean(temp, var, "bool_covid_after")
# text offset
y_valid = temp[var].dropna()
if len(y_valid) > 0:
    y_span = y_valid.max() - y_valid.min()
    offset = 0.03 * y_span if y_span > 0 else 1
else:
    offset = 1
def add_mean_segment(ax, start, end, mean_value, label):
    if pd.notna(mean_value):
        ax.plot([start, end], [mean_value, mean_value],
            color="black", linewidth=1.0, solid_capstyle="butt"
                )
        mid = start + (end - start) / 2
        ax.text(mid, mean_value + offset, label, ha="center", va="bottom",
            fontsize=10, color="black" )
# gfc
add_mean_segment(ax, gfc_before_start, gfc_before_end, mean_gfc_before, "before")
add_mean_segment(ax, gfc_start, gfc_end, mean_gfc_during, "gfc")
add_mean_segment(ax, gfc_after_start, gfc_after_end, mean_gfc_after, "after")
# covid
add_mean_segment(ax, covid_before_start, covid_before_end, mean_covid_before, "before")
add_mean_segment(ax, covid_start, covid_end, mean_covid_during, "covid")
add_mean_segment(ax, covid_after_start, covid_after_end, mean_covid_after, "after")
# title and axes
ax.set_title(var, fontsize=12)
ax.set_xlabel("time", fontsize=10)
ax.set_ylabel(var, fontsize=10)
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
ax.tick_params(axis="x", rotation=45)
ax.yaxis.set_major_formatter(StrMethodFormatter("{x:,.0f}"))
ax.grid(True, linestyle="--", alpha=0.4)
for j in range(len(variables_to_plot), len(axes)):
    axes[j].set_visible(False)
plt.tight_layout()
plt.savefig("mon_graph4.png", dpi=300, bbox_inches="tight")
plt.show()

```

R codes for structure and regressions

```

rm(list=ls())
# erase all objects from memory
graphics.off()
# close all open graphics
install.packages("scatterplot3d")
# install 3D scatter package
install.packages("plotly")
# install plotly package
library(scatterplot3d)
# load 3D scatter package

```

```

library(scatterplot3d)                                # load 3D scatter package
library(plotly)
#setwd(dirname(rstudioapi::getActiveDocumentContext()$path)) # set working directory to script location
attach(data1)
getwd()
setwd("C:/Users/Dell/Desktop/Market Microstructure/Summarative/Question")
data1 <- read.csv("medium_bucket.csv")                # load the dataset
# Principal Components Analysis #####
head(data1)                                           # preview first rows of the dataset
apply(data1, 2, mean)                                # compute column means
apply(data1, 2, sd)                                  # compute column standard deviations
names(data1)                                          # display variable names
par(mfrow = c(3, 3))                                # prepare multi-panel plotting area
hist(bid_to_cover,
xlab = "bid_to_cover",
main = NULL,
breaks = 25,
probability = TRUE)                                # draw histogram of bid_to_cover
lines(density(data1$bid_to_cover, na.rm = TRUE, adjust = 3), lwd = 2) # add density curve
graphics.off()                                       # close graphics device
##### PCA ANALYSIS
pca_vars <- c("bid_amount",
"amount_served",
"bid_to_cover",
"outstanding_notional",
"Vix_euro",
"auction_yield",
"weighted_average_price",
"benchmark_yield",
"auction_premium_bp",
"shock_log_dev")
data1 <- data1[, pca_vars, drop = FALSE]
data1 <- na.omit(data1)
pr.out <- prcomp(data1, scale. = TRUE)
pr.out <- prcomp(data1, scale = TRUE, xlab = rep("", nrow(pr.out$x))) # obtaining principal components
pr.out$center                                         # averages (same as line 18)
pr.out$scale                                          # std. dev. (same as line 19)
pr.out$rotation                                       # principal component loadings
pr.out$x                                              # principal components
round(scale(data1), digits = 2)                      # rescaled data is the same as...
round(pr.out$x%*%t(pr.out$rotation),
digits = 2)                                          # principal comp. factorization
graphics.off()
biplot(pr.out,
scale = 0,

```

```

xlabs = rep("", nrow(pr.out$x))
pr.out$sdev^2                                # variances of principal components
diag(t(pr.out$x)%*%pr.out$x/49)             # ... obtained differently
pve <- pr.out$sdev^2/sum(pr.out$sdev^2)      # prop. of var. due to each p.c.
par(mfrow = c(1, 2))                        # preparing Figure 12.3
plot(pve, xlab = "Principal Component",
ylab = "Proportion of Variance Explained", ylim = c(0, 1),
type = "b")                                # left panel
plot(cumsum(pve), xlab = "Principal Component",
ylab = "Cumulative Proportion of Variance Explained",
ylim = c(0, 1), type = "b")
### 3D Modifiable graph
x <- data1[, pca_vars, drop = FALSE]         # keep only selected PCA variables
x <- na.omit(x)                             # remove missing values
pr.out <- prcomp(x, scale. = TRUE)           # run PCA for 3D visualization
load <- pr.out$rotation[, 1:3]               # keep loadings of first 3 components
fig <- plot_ly()                             # initialize empty 3D figure
for(i in 1:nrow(load)) {fig <- fig %>% add_trace(x = c(0, load[i,1] * 3),
y = c(0, load[i,2] * 3), z = c(0, load[i,3] * 3),type = "scatter3d",
mode = "lines+text", line = list(color = "red", width = 5),
text = c("", rownames(load)[i]), showlegend = FALSE)      # add one loading vector at a time
}
fig                                           # display interactive 3D plot
##### REGRESSIONS
rm(list = ls())                             # clear all objects from memory
graphics.off()                              # close all open graphics
install.packages("AER")                     # install AER package
install.packages("plm")                     # install plm package
install.packages("lmtest")                  # install lmtest package
install.packages("sandwich")                # install sandwich package
install.packages("stargazer")               # install stargazer package
install.packages("AER")                     # install AER again (duplicate line kept)
install.packages("gam")                     # install gam package
library(splines)                            # load splines package
library(gam)                                # load gam package
library(AER)                                # load AER package
library(stargazer)                          # load stargazer package
library(plm)                                # load plm package
library(ggplot2)                            # load ggplot2 package
library(quantreg)                           # load quantile regression package
library(lmtest)                             # load lmtest package
library(sandwich)                           # load sandwich package
tau_min <- 0.05                             # minimum quantile
tau_max <- 0.95                             # maximum quantile
tau_step <- 0.05                            # quantile step

```

```

taus <- seq(tau_min, tau_max, by = tau_step) # sequence of quantiles
setwd(dirname(rstudioapi::getActiveDocumentContext()$path)) # set working directory to script folder
df <- read.csv("data1.csv") # load dataset
##### PLOT QUANTILE AND OLS
# estimate the models
df$auction_date <- as.Date(df$auction_date, format = "%m/%d/%Y") # convert date variable
names(df) # display variable names
mod_ols <- lm(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, data = df) # OLS model
mod_q25 <- rq(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, tau = 0.25, data = df) #
mod_q50 <- rq(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, tau = 0.50, data = df) #
mod_q75 <- rq(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, tau = 0.75, data = df) #
par(mar = c(4,4,2,1)) # set plot margins
par(mfrow = c(1,1)) # single plot window
# scatter plot
plot(df$shock_log_dev, df$auction_premium_bp,
xlab = "shock_log_dev",
ylab = "auction_premium_bp",
main = "OLS and quantile regression",
pch = 1) # draw scatter plot
# build prediction grid for quantile lines
x_grid <- seq(min(df$shock_log_dev, na.rm = TRUE),
max(df$shock_log_dev, na.rm = TRUE),
length.out = 200) # first prediction grid
#####
x_grid <- seq(min(df$shock_log_dev, na.rm = TRUE),
max(df$shock_log_dev, na.rm = TRUE),
length.out = 300) # final prediction grid
new_df <- data.frame(
shock_log_dev = x_grid,
Vix_euro = mean(df$Vix_euro, na.rm = TRUE),
outstanding_notional = mean(df$outstanding_notional, na.rm = TRUE),
auction_date = rep(mean(df$auction_date, na.rm = TRUE), length(x_grid))
) # new data holding controls at their means
y_q25 <- predict(mod_q25, newdata = new_df) # predicted q25 values
y_q75 <- predict(mod_q75, newdata = new_df) # predicted q75 values
y_ols <- predict(mod_ols, newdata = new_df) # predicted OLS values
# add fitted lines
lines(x_grid, y_q25, lwd = 2, lty = 2, col = "blue") # add q25 line
lines(x_grid, y_q75, lwd = 2, lty = 2, col = "red") # add q75 line
lines(x_grid, y_ols, lwd = 2, lty = 1, col = "black") # add OLS line
# add legend
legend("topleft",
legend = c("ols", "q25", "q75"),
col = c("black", "blue", "red"),
lwd = 2,

```

```

lty = c(1, 2, 2),
bty = "n")                                # display legend
##### PLOT RESIDUALS
res_lin <- residuals(mod_ols)                # extract OLS residuals
fit_lin <- fitted(mod_ols)                  # extract OLS fitted values
plot(fit_lin, res_lin,
xlab = "fitted values",
ylab = "residuals",
main = "residuals vs fitted - linear model",
pch = 19)                                # residual plot
abline(h = 0, lty = 2)                    # horizontal reference line
##### HETEROSCEDASTICITY TEST
bptest(mod_ols)                            # Breusch-Pagan test
bptest(mod_ols, studentize = TRUE)         # studentized Breusch-Pagan test
coeftest(mod_ols, vcov = vcovHC(mod_ols, type = "HC1")) # robust standard errors
## REGRESSION
# estimate the models again for regression table
mod_q25 <- rq(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, tau = 0.25, data = df) #
mod_q75 <- rq(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, tau = 0.75, data = df) #
mod_ols <- lm(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional, data = df)
mod_olsd <- lm(auction_premium_bp ~ shock_log_dev + Vix_euro + outstanding_notional + auction_date, data = df) #
mod_olsd2 <- lm(auction_premium_bp ~ shock_log_dev + I(shock_log_dev^2) + Vix_euro +
outstanding_notional + auction_date, data = df) # quadratic OLS with date
mod2_q25 <- rq(auction_premium_bp ~ shock_log_dev + I(shock_log_dev^2) + Vix_euro +
outstanding_notional + auction_date, tau = 0.25, data = df) # quadratic q25
mod2_q75 <- rq(auction_premium_bp ~ shock_log_dev + I(shock_log_dev^2) + Vix_euro +
outstanding_notional + auction_date, tau = 0.75, data = df) # quadratic q75
mod_olsr <- coeftest(mod_ols, vcov = vcovHC(mod_ols, type = "HC1")) # robust OLS without date
mod_olsdr <- coeftest(mod_olsd, vcov = vcovHC(mod_olsd, type = "HC1")) # robust OLS with date
mod_olsd2r <- coeftest(mod_olsd2, vcov = vcovHC(mod_olsd2, type = "HC1")) # robust quadratic OLS
stargazer(mod_q25, mod_q75, mod_olsr, mod_olsdr, mod_olsd2r, type = "text") # regression output table
summary(mod_ols)$r.squared                  # OLS R-squared
summary(mod_ols)$adj.r.squared              # OLS adjusted R-squared
summary(mod_olsd)$r.squared                 # OLS with date R-squared
summary(mod_olsd)$adj.r.squared             # OLS with date adjusted R-squared
summary(mod_olsd2)$r.squared                # quadratic OLS R-squared
summary(mod_olsd2)$adj.r.squared            # quadratic OLS adjusted R-squared
##### BEST POLYNOMIALS
library(boot)                              # load boot package for cross-validation
# prepare a clean dataset for polynomial selection
df2 <- df[, c("auction_premium_bp", "shock_log_dev", "Vix_euro", "outstanding_notional", "auction_date")] # keep needed var
df2 <- na.omit(df2)                        # remove missing values
set.seed(14)                              # set seed for reproducibility
k <- nrow(df2)                             # use LOOCV with one fold per observation
max_deg <- 6                              # maximum polynomial degree teste

```

```

mse_cv <- numeric(max_deg) # store cross-validation errors
names(df) # display variable names
for(d in 1:max_deg){form <- as.formula( paste0("auction_premium_bp ~
poly(shock_log_dev, ", d, ", raw=TRUE) + Vix_euro + outstanding_notional + auction_date"))
mod <- glm(form, data = df2) # fit model for degree d
cv_res <- cv.glm(df2, mod, K = k) # compute LOOCV error
mse_cv[d] <- cv_res$delta[1] # save CV MSE
}

best_deg <- which.min(mse_cv) # degree with minimum CV error
cat("meilleur degré =", best_deg, "\n") # print best degree
cat("mse minimum =", mse_cv[best_deg], "\n") # print minimum MSE
plot(1:max_deg, mse_cv,
type = "b",
pch = 19,
xlab = "Polynomial degree of shock_log_dev",
ylab = "LOOCV MSE",
main = "Best model premia-shock_log_dev") # plot CV error by degree
abline(v = best_deg, lty = 2) # add vertical line for best degree
##### POLYNOMIAL 2
poly25 <- rq(auction_premium_bp ~ poly(shock_log_dev, 2, raw = TRUE)
+ Vix_euro + outstanding_notional + auction_date, tau = 0.25, data = df2) # quadratic q25 model
poly75 <- rq(auction_premium_bp ~ poly(shock_log_dev, 2, raw = TRUE)
+ Vix_euro + outstanding_notional + auction_date, tau = 0.75, data = df2) # quadratic q75 model
mod_ols <- lm(auction_premium_bp ~ poly(shock_log_dev, 2, raw = TRUE) + Vix_euro
+ outstanding_notional + auction_date, data = df) # quadratic OLS model
mod_olsr <- coeftest(mod_ols, vcov = vcovHC(mod_ols, type = "HC1")) # robust standard errors for quadratic OLS
stargazer(poly25, poly75, mod_olsr, type = "text") # display quadratic model results
##### PLOT IMAGE DISTRIBUTION IMAGE
beta_vec <- numeric(length(taus)) # store estimated coefficients
low_vec <- numeric(length(taus)) # store lower bounds
high_vec <- numeric(length(taus)) # store upper bounds
# loop across quantiles (some one can change the coefficient for all plots)
eta_vec <- numeric(length(taus)) # extra vector kept as in original code
low_vec <- numeric(length(taus)) # reset lower bounds
high_vec <- numeric(length(taus)) # reset upper bounds
df$auction_date <- as.Date(df$auction_date, format = "%m/%d/%Y") # ensure date format
for (i in seq_along(taus)) {
tau_i <- taus[i] # current quantile
model_i <- rq(auction_premium_bp ~ shock_log_dev + I(shock_log_dev^2) +
Vix_euro + outstanding_notional + auction_date, tau = tau_i, data = df) # fit quantile model
sum_i <- summary(model_i) # summarize current model
coef_mat <- sum_i$coefficients # extract coefficient matrix
row_id <- which(rownames(coef_mat) == "shock_log_dev") # locate linear term
beta_vec[i] <- coef_mat[row_id, 1] # coefficient estimate
low_vec[i] <- coef_mat[row_id, 2] # lower bound or standard error column

```

```

    high_vec[i] <- coef_mat[row_id, 3] }
# final results table
results_qr <- data.frame(tau    = taus, beta  = beta_vec, lower = low_vec, upper = high_vec)
print(results_qr)                # print quantile regression results
# coefficient plot across quantiles
plot(results_qr$tau, results_qr$beta, type = "b", pch = 19, lwd = 2, xlab = "quantile",
      ylab = "coefficient on linear term")    # plot coefficient path over quantiles

```